ÉTUDE DE CAS 02 / LOGICIEL WINDOWS

ORBIT 2.0

Concevoir une application desktop de diagnostic matériel

CARLOS DA COSTA

ÉTUDE DE CAS APPROFONDIE · ÉDITION REVUE ET AUGMENTÉE · OCTOBRE 2026

Electron, React, TypeScript, C#, API Windows, télémétrie locale, sécurité IPC et packaging.

Note de l’auteur

J’analyse ici le passage d’un tableau de bord local à un véritable logiciel Windows. L’enjeu n’était pas seulement d’améliorer l’interface : il fallait établir une architecture desktop, obtenir des informations matérielles sans les inventer, maîtriser la collecte et distribuer un exécutable vérifiable.

Le périmètre principal est celui du dépôt public orbit-windows et de ses dossiers d’architecture, d’audit et de validation. Les résultats de tests matériels datés sont attribués au rapport du projet ; je ne présente pas les fixtures unitaires comme des essais physiques réalisés sur tous les composants possibles.

Méthode de lecture

Je présente les raisonnements de conception et les arbitrages techniques à partir des fichiers et tests vérifiables du projet. Les sections distinguent ce qui est **implémenté dans le code**, ce qui a été **validé par des tests**, et ce qui reste **à confirmer sur les services réels**. La première personne décrit mon appropriation et mon analyse du travail ; elle ne prétend pas prouver que chaque ligne de code a été écrite sans assistance.

Sommaire détaillé

01 Partir d'un prototype et reformuler l'objectif 3

02 Comparer l'ancienne architecture à Orbit 2.0 4

03 Concevoir un protocole fiable entre Electron et C# 5

04 Modéliser l'information : valeur, état, source 6

05 Obtenir les mesures CPU, GPU et mémoire 7

06 Écrans, stockage, réseau et audio 8

07 Périphériques 2,4 GHz : la difficulté de l'identification 9

08 Performance : éviter qu'un moniteur ralentisse le PC 10

09 Sécuriser une application Electron 11

10 Expérience utilisateur et personnalisation 12

11 Problèmes concrets et corrections défendables 13

12 Tests et résultats sans surinterprétation 16

13 Construire et distribuer une application Windows 17

14 Bilan personnel et décisions d'ingénierie 18

15 Transformer le besoin en exigences vérifiables 19

16 Contrats TypeScript : rendre l'incertitude explicite 20

17 Collecte native C# et supervision du processus 21

18 Séries temporelles et performance du moniteur 22

19 Périphériques réels : identification imparfaite et regroupement 23

20 Menaces propres aux applications Electron 24

21 Construction Windows : de la source à un artefact utilisable 25

22 Programme d'essais et journal de décisions 26

23 Questions d'oral et réponses argumentées 27

24 Annexes techniques et documentation 28

01 Partir d’un prototype et reformuler l’objectif

Pourquoi un dashboard web ne suffisait plus

Le point de départ documenté est un prototype comportant un serveur local Node, des pages HTML/CSS/JavaScript et des scripts PowerShell interrogeant Windows. Il permettait de réunir certaines informations, mais il conservait des contraintes propres à une page web : lancement d’un navigateur, dépendance à un serveur HTTP local et interactions système fragmentées.

Je voulais que le programme se comporte comme une application Windows : ouverture autonome, navigation cohérente, collecte en arrière-plan sous contrôle de l’utilisateur, préférences persistantes, zone de notification et installateurs. J’ai donc redéfini l’architecture plutôt que de retoucher uniquement les styles.

Un cahier des charges centré sur l’usage

- Afficher une vue d’ensemble synthétique : CPU, GPU, mémoire, stockage, réseau et écrans lorsque les données existent.
- Séparer mesures instantanées et informations relativement stables de l’inventaire.
- Proposer des pages matériel, performance, périphériques, écrans, disques, réseau, audio, profil, concentration et réglages.
- Conserver une interface moderne, lisible, avec navigation rapide, thèmes et prise en charge du clavier.
- Ne pas afficher de niveau de batterie fictif pour les périphériques 2,4 GHz lorsqu’aucune API fiable ne le fournit.
- Limiter les privilèges, l’activité en arrière-plan et la collecte d’informations identifiantes.
- Permettre un build d’installation et une version portable sur Windows x64.

La contrainte fondamentale : une donnée peut être inconnue

Un diagnostic utile ne doit pas confondre « impossible à lire », « non présent » et « zéro ». J’ai traité cette distinction comme un principe de produit. Une température indisponible ne devient pas 0 °C ; une souris branchée via un récepteur USB ne reçoit pas automatiquement une batterie à 100 %.

Cette exigence a des conséquences sur les types TypeScript, les normalizeurs, les tests et le rendu des graphiques : chaque métrique doit porter un état et une provenance, et les courbes doivent montrer leurs lacunes plutôt que les masquer.

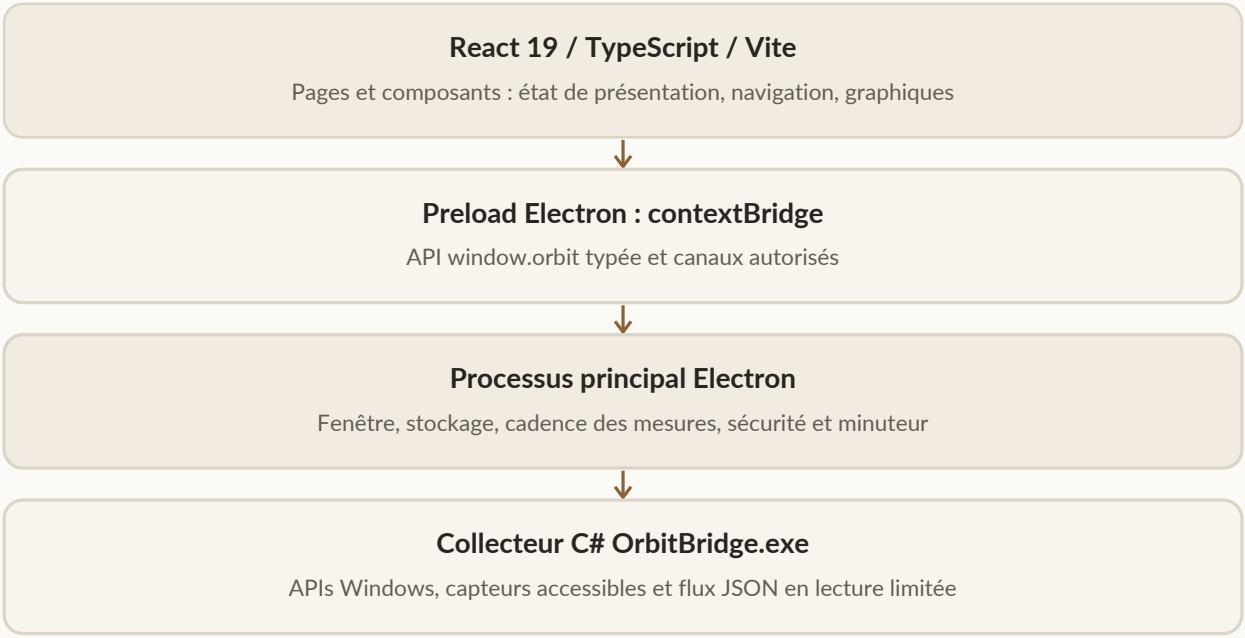
> **Décision fondatrice**
>
> J’ai choisi une application desktop locale plutôt qu’une page web qui simule un logiciel. L’interface apporte l’ergonomie ; le processus principal et le collecteur natif assurent le dialogue contrôlé avec Windows.

02 Comparer l’ancienne architecture à Orbit 2.0

La réorganisation du code

Le document **docs/AUDIT.md** rapproche les douze fichiers d’un prototype initial des composants de la nouvelle version. Le serveur local, le script de capture système, le script audio et le JavaScript de page monolithique sont remplacés par des responsabilités distinctes. Les sources versionnées conservent une traçabilité de cette transition.

Le plus important n’est pas l’adoption d’un framework pour lui-même : c’est de diminuer les dépendances cachées entre l’affichage, la collecte et les actions système.



Rôle des couches

Couche	Ce que j'y place
Renderer React	Affichage et interactions, sans accès libre à Node ou aux fichiers.
Preload	Petite interface publique window.orbit ; aucun IPC général exposé.
Electron main	Validation des appels, gestion de fenêtre, préférences, orchestration et cycles de collecte.
C# natif	Interrogation d'API spécifiques Windows, normalisation initiale et commandes bornées.
Tests et CI	Tests TypeScript/Node, tests natifs, compilation Windows et artefacts.

Pourquoi React et TypeScript ?

La nouvelle interface est répartie entre des pages et des composants réutilisables. Les composants peuvent être chargés à la demande ; les états de préférences, d'inventaire et de mesures sont séparés. TypeScript sert surtout à formaliser les contrats : quelles données peuvent manquer, quelle commande accepte quelle entrée et quel état doit être retourné.

La navigation ne doit pas reconstruire toute l'interface à chaque mise à jour de température. Cette dissociation permet aussi de diminuer le nombre de composants abonnés aux données les plus fréquemment modifiées.

03 Concevoir un protocole fiable entre Electron et C#

Pourquoi un processus C# persistant ?

Le prototype historique utilisait notamment PowerShell pour interroger Windows et manipuler le son, avec un coût potentiel de lancement et de compilation répété. La version 2.0 dispose d'un exécutable .NET dédié, **OrbitBridge.exe**, dont le processus principal pilote les appels.

Le protocole passe par **stdin/stdout en JSON**. Ce choix simplifie la séparation technique : le renderer n'a pas besoin d'ouvrir lui-même les objets COM, WMI ou HID. Electron contrôle les demandes et peut limiter leur fréquence, leur taille ou leur durée.

Définir un contrat plutôt qu'un shell

Les commandes natives correspondent à des opérations explicites telles que l'inventaire, l'échantillonnage rapide ou certains contrôles audio. Elles ne transportent pas une chaîne PowerShell arbitraire fournie par l'interface. Le backend valide également les paramètres des actions reçues.

Le module **system/providers/bridge.ts** introduit une file de demandes et des mécanismes d'échec. Les limites de durée et de requêtes en attente servent à éviter qu'un collecteur bloqué accumule indéfiniment du travail.

Gérer l'indisponibilité du collecteur

Une interface ne doit pas s'effondrer si une commande native échoue. Le composant **Monitor** conserve la possibilité de lire certaines informations par Node, notamment CPU et mémoire, puis annote les mesures dépendantes du bridge avec un état temporairement indisponible.

L'interface expose la distinction entre capacité présente, mesure non disponible et erreur. Ce choix rend l'échec observable sans fabriquer des valeurs trompeuses.

```
Interface React -> window.orbit.sample()
Preload -> canal IPC explicitement autorisé
Electron main -> vérification de la fenêtre et des paramètres
NativeBridge -> requête JSON avec identifiant et délai max
OrbitBridge.exe -> consultation API Windows
Retour -> valeurs + source + état de disponibilité
```

04 Modéliser l'information : valeur, état, source

La métrique comme objet explicable

Dans **shared/logic.ts**, une métrique ne représente pas seulement un nombre. Elle inclut une valeur éventuellement nulle, un état, un fournisseur, une date de mise à jour et, si nécessaire, une unité. Les valeurs mesurées deviennent ainsi plus simples à interpréter et à tester.

Une batterie non prise en charge est représentée par **null** et un état d'indisponibilité. Un taux de réseau sans compteur précédent n'est pas remplacé par un zéro inventé. Pour l'utilisateur, cette honnêteté est plus utile qu'une interface apparemment complète mais inexacte.

Inventaire versus mesures temps réel

J'ai séparé les informations relativement durables — processeur, modules de mémoire, volumes, périphériques, écrans — des données qui changent souvent — usage CPU, trafic réseau, activité disque ou volume audio.

Le code **system/monitor.ts** possède des verrous distincts pour l'inventaire et les mesures rapides. Il peut rafraîchir l'inventaire périodiquement ou à la suite d'un événement matériel sans le recalculer à chaque seconde.

Quand une courbe doit être interrompue

Les historiques et graphiques ont une taille bornée. Une mesure nulle ne doit pas être reliée automatiquement à la suivante comme si l'intervalle avait été observé. Ce principe évite de créer de fausses tendances et facilite l'analyse de capteurs occasionnellement indisponibles.

Je retiens que la visualisation est aussi une forme de traitement des données : choisir de relier deux points, ou de laisser une rupture, constitue déjà une décision technique.

> **Exemple à expliquer**
>
> Un affichage de température « non disponible » peut être un résultat correct. Il signale qu'aucune source permise n'a fourni une valeur défendable. Le problème serait de transformer cette absence en chiffre certain.

05 Obtenir les mesures CPU, GPU et mémoire

CPU : calculer une utilisation différentielle

Pour le CPU, la fonction **cpuDelta** compare deux séries de compteurs cumulés fournis par le système. Pour chaque cœur, elle calcule la différence de temps total et de temps inactif entre deux échantillons ; l'utilisation se rapproche de **100 × (1 − temps_inactif / temps_total)**.

Ce calcul exige un intervalle réel. Si les compteurs sont absents, incohérents ou réinitialisés, la mesure doit être traitée comme temporairement indisponible plutôt que comme une activité négative ou supérieure à 100 %. La valeur finale est bornée entre 0 et 100.

```
delta_total = total_après - total_avant
delta_idle = idle_après - idle_avant
si delta_total <= 0 : état = indisponible
sinon utilisation = 100 * (1 - delta_idle / delta_total)
utilisation bornée dans [0, 100]
```

RAM : capacité, usage et caractéristiques des barrettes

Les mesures de mémoire globale s'appuient sur les informations du système. Pour les barrettes physiques, le collecteur utilise notamment WMI et des repli SMBIOS lorsque certaines classes ne répondent pas. Le document matériel distingue correctement **MT/s** et **MHz** ; il ne déduit pas des timings CAS non accessibles.

Le rapport de validation mentionne un essai réel ayant récupéré deux barrettes DDR4 de 16 Go à 3 200 MT/s. Ce résultat est propre au matériel testé ; le logiciel doit rester prudent lorsque d'autres BIOS ne publient pas le même niveau de détail.

GPU : identifier le fournisseur et les limites

L'identité et certaines caractéristiques GPU viennent de DXGI ; les relevés NVIDIA peuvent compléter ces données par NVML, et les compteurs PDH servent à certains usages GPU et mémoire. Une source de mémoire graphique ancienne ou tronquée ne doit pas être considérée comme une mesure fiable pour des cartes modernes.

Le collecteur distingue plusieurs GPU et refuse les associations ambiguës entre lignes de compteurs et cartes réelles. L'interface ne doit pas attribuer au mauvais GPU une température ou un taux d'utilisation simplement pour remplir toutes les cases.

Capteurs CPU et capteurs constructeur

Certaines températures ne sont accessibles que lorsqu'un logiciel ou un espace WMI de capteurs les expose déjà. Orbit peut utiliser des capteurs de LibreHardwareMonitor présents, mais ne télécharge pas ni n'installe automatiquement de pilote ou de DLL pour contourner l'absence de données.

J'ai privilégié le diagnostic sans élévation injustifiée plutôt qu'une quête de données exhaustives à n'importe quel coût.

06 Écrans, stockage, réseau et audio

Écrans et fréquence réelle

L'API **QueryDisplayConfig** et les méthodes de lecture de modes d'écran permettent de rapporter une fréquence active parfois fractionnaire. Le rapport du projet indique un essai à 239,765 Hz, puis un autre à 241 Hz. Cette précision évite de remplacer systématiquement un résultat par un nom commercial comme « 240 Hz ».

Les informations EDID, HDR et fréquence préférée restent limitées par ce que Windows et le moniteur exposent. Une fréquence active n'est pas une garantie du maximum physique de la dalle ; le type OLED ou IPS ne doit pas être inventé.

Stockage : retrouver un disque malgré les lacunes WMI

Le code combine les classes de stockage Windows avec des accès en lecture seule comme **IOCTL_STORAGE_QUERY_PROPERTY**, les volumes .NET et les correspondances d'étendues. L'objectif est de rapprocher disques physiques, partitions et volumes sans les confondre.

Le rapport d'essai note que deux disques et deux volumes ont pu être retrouvés malgré des refus WMI. Ce type de repli accroît la robustesse, mais il ne transforme pas une estimation de santé fournie par le pilote en véritable diagnostic SMART/NVMe complet.

Réseau : raisonner sur des deltas

Le débit réseau doit être calculé à partir d'une différence de compteurs divisée par le temps écoulé. Si l'interface est réinitialisée, si l'identifiant change ou si les compteurs diminuent, le taux ne peut pas être interprété normalement.

La fonction **networkRates** vérifie la validité des compteurs et du temps entre deux observations. Elle signale l'indisponibilité temporaire dans les scénarios ambigus au lieu de produire un débit négatif.

Audio : employer les interfaces publiques

Le composant natif utilise les interfaces **Core Audio COM** pour lire et, dans les limites du contrat, modifier le volume ou l'état muet. Les sorties détectées sont réelles et la sélection de sortie reste une responsabilité des paramètres Windows.

Le choix de ne pas utiliser une interface privée comme PolicyConfig évite de dépendre d'un comportement non garanti. Les commandes audio sont validées dans une plage explicite : volume entier de 0 à 100 et état muet booléen.

07 Périphériques 2,4 GHz : la difficulté de l'identification

Un même appareil peut exposer plusieurs interfaces

Un clavier ou une souris USB/HID peut apparaître plusieurs fois dans l'inventaire, notamment en cas de macros, récepteur radio et interfaces composites. Grouper uniquement par nom fusionnerait parfois deux appareils distincts ; conserver chaque interface comme appareil séparé gonflerait artificiellement la liste.

La fonction **groupDevices** privilégie le **ContainerId** Windows lorsqu'il est valable, puis utilise des identifiants de parent ou d'instance comme repli. Elle sélectionne une description significative en pénalisant les intitulés génériques du type « périphérique USB composite ».

Connexion détectée versus interprétation commerciale

La présence d'une interface USB ne signifie pas nécessairement que la souris est câblée : elle peut correspondre à un dongle. La détection du fournisseur et du produit peut aider, mais ne suffit pas à certifier une variante commerciale précise.

Le rapport matériel évoque un clavier G-LAB et une souris Pulsar dont les interfaces radio ne fournissent pas de contrat public fiable pour lire les batteries. Plutôt que d'implémenter une commande expérimentale, l'interface propose un état « batterie non communiquée » et des réglages manuels clairement distingués des données mesurées.

D'où vient une batterie de périphérique ?

La lecture s'appuie sur les propriétés Windows ou sur des usages HID standard comme **Battery Strength** quand ils sont déclarés par le périphérique. Le code n'émet pas de rapports HID de sortie et ne tente pas d'écrire un firmware.

Le choix est révélateur d'une démarche d'ingénierie responsable : un affichage plus complet ne justifie pas un protocole propriétaire non documenté potentiellement dangereux.

> **Le compromis assumé**
>
> Mieux vaut afficher « non renseigné » pour une souris 2,4 GHz que de faire apparaître un pourcentage inventé. La qualité d'un outil système tient à l'exactitude de ses limites.

08 Performance : éviter qu'un moniteur ralentisse le PC

Une cadence explicite et adaptable

Une application de monitoring consomme elle-même des ressources. Pour éviter une boucle inutile, les préférences définissent un intervalle de **1, 3, 5 ou 10 secondes**, avec 3 secondes par défaut. Lorsque la fenêtre est masquée et que le mode économie est actif, l'intervalle peut passer à une minute. En veille, la collecte est suspendue.

Les fonctions **effectiveInterval** et **Monitor.schedule** formalisent ces décisions, afin que le code de l'interface n'ait pas à interpréter directement les transitions d'alimentation et de visibilité.

Prévenir les chevauchements

Les indicateurs fastBusy et inventoryBusy évitent qu'un deuxième échantillonnage ou inventaire démarre alors que le précédent n'est pas terminé. Cela réduit les accumulations de demandes sur des API lentes comme WMI.

Au réveil, les anciennes mesures CPU et réseau sont réinitialisées avant de calculer un nouveau débit. Utiliser des différences de compteurs accumulées avant et après une veille pourrait créer des pics absurdes.

Conserver uniquement l'historique utile

Le projet borne les courbes à un nombre limité de points. L'historique facultatif est également limité dans le temps et dans son volume. Les stores de l'interface séparent les mesures rapides des préférences et des données de navigation.

Cette architecture favorise une consommation prévisible plutôt qu'une croissance silencieuse de mémoire à chaque minute de fonctionnement.

Démarrage en arrière-plan

Quand l'application démarre masquée avec Windows, le collecteur peut différer l'inventaire détaillé jusqu'à l'ouverture de la fenêtre. Cela évite de solliciter WMI et d'autres API uniquement pour maintenir une icône dans la zone de notification.

Le problème résolu n'est donc pas qu'une optimisation abstraite : c'est préserver le confort d'utilisation du PC qu'Orbit est censé surveiller.

09 Sécuriser une application Electron

Le navigateur embarqué n'est pas un processus privilégié

Electron associe un moteur web à des capacités natives. C'est puissant, mais une erreur de frontière pourrait donner à un élément rendu dans la fenêtre davantage de privilèges que nécessaire. Le projet configure notamment **contextIsolation: true, nodeIntegration: false, sandbox: true et webSecurity: true**.

Le renderer est servi depuis une origine locale **orbit://app/**, et les connexions HTTP/HTTPS/WebSocket provenant de cette interface sont refusées. Les webviews, nouvelles fenêtres et navigations distantes sont également contraintes dans l'architecture documentée.

Vérifier l'origine et la frame d'un appel

Le code **shared/security.ts** vérifie que l'appel IPC provient d'une fenêtre appartenant à l'application, de la frame principale, avec le protocole et l'hôte attendus. Il refuse les URL contenant des éléments inattendus.

C'est essentiel : exposer une fonction par le preload ne suffit pas à sécuriser son exécution si un autre contenu peut obtenir la même capacité. Les chemins demandés sont également résolus dans un dossier borné et seuls certains types de ressources sont acceptés.

Valider les préférences et les commandes

La fonction **validatePreferences** refuse les clés inconnues, les intervalles non prévus, les catégories invalides et les structures excessives. Les identifiants de périphériques et noms personnalisés ont des contraintes explicites.

Cette validation protège aussi la stabilité : un JSON corrompu ou une préférence incohérente peut casser le lancement d'une application si elle est relue sans contrôle. Le magasin de préférences utilise une écriture temporaire puis un renommage pour limiter les fichiers partiellement écrits.

Journaux et vie privée

Les diagnostics peuvent contenir des informations identifiantes. Orbit limite les logs aux codes de fournisseur et aux états, sans adresses réseau, noms de personnes ou identifiants matériels. Le fichier local est borné en taille et les répétitions identiques peuvent être dédupliquées.

Le programme n'est pas conçu comme un service de collecte cloud : il ne demande pas de compte Orbit, ne publie pas de télémétrie et conserve par défaut les traitements en local. Il faut néanmoins vérifier les exécutables finaux et les dépendances avant une distribution large.

10 Expérience utilisateur et personnalisation

Des pages spécialisées plutôt qu'un écran saturé

Le code rassemble les vues dans **src/pages/** : Overview, Performance, Hardware, Devices, Displays, Storage, Network, Audio, Account, Focus et Settings. Cette répartition rend la navigation lisible et permet de charger des pages à la demande.

L'utilisateur peut commencer par une synthèse puis inspecter une catégorie. La recherche rapide trie les résultats selon la présence des mots dans le titre, la description et les mots-clés, avec une normalisation des accents.

Thèmes et états locaux

Les préférences encadrent les thèmes système, clair ou sombre, certains accents colorés, la compacité, les animations réduites et la disposition des panneaux. Les personnalisations de périphériques servent à améliorer la lisibilité, mais ne sont pas présentées comme une identification matérielle certifiée.

Un nom défini par l'utilisateur et une description obtenue depuis un descripteur HID n'ont pas la même provenance. Le modèle doit conserver cette distinction pour que l'interface reste honnête.

Photo de profil Windows

Le logiciel recherche des informations locales comme le nom de compte et, si elle est accessible, la photo mise en cache par Windows. Il peut proposer un avatar personnalisé. Il n'utilise pas de récupération non autorisée par OAuth et ne demande pas le mot de passe Microsoft.

Le rapport de validation indique précisément que la photo du compte n'a pas pu être lue dans l'environnement testé. Cette limite est utile à présenter : le cas non pris en charge a été identifié plutôt que masqué.

Le minuteur de concentration

Un minuteur lié à un composant React risquerait de perdre son état lorsque la page se ferme. Le code **FocusTimer** conserve une échéance temporelle dans le processus principal et prévoit les commandes de démarrage, pause, réinitialisation et durée.

Le temps restant découle de l'échéance, non d'un décrément graphique supposé exact à chaque seconde. L'interface peut donc être cachée sans que le modèle de temps dépende de son rythme de rendu.

11 Problèmes concrets et corrections défendables

Le prototype ouvrait un navigateur

Contexte et problème. Le projet ressemblait davantage à un tableau de bord local qu'à un logiciel Windows.

Analyse. Un serveur HTTP actif et un lanceur de navigateur compliquaient l'expérience et la sécurité.

Réponse technique. J'ai adopté Electron, supprimé le serveur actif et fait passer les actions par une API preload contrôlée.

Vérification. Les tests IPC et l'audit de migration décrivent la nouvelle frontière.

Ce que je retiens. Le fonctionnement sur Windows doit toujours être confirmé avec des installations réelles.

Plusieurs périphériques portaient le même nom

Contexte et problème. Deux appareils pouvaient être fusionnés, tandis qu'un appareil composite apparaissait plusieurs fois.

Analyse. Le nom d'affichage n'est pas un identifiant stable ni unique.

Réponse technique. Regroupement par ContainerId, repli sur le parent/instance et sélection d'un descripteur pertinent.

Vérification. Tests de regroupement HID, noms identiques et appareils déconnectés.

Ce que je retiens. Certains récepteurs radio restent impossibles à identifier au modèle commercial exact.

Une batterie de souris était indisponible

Contexte et problème. Le dongle 2,4 GHz exposait un périphérique sans niveau de charge standard exploitable.

Analyse. Un pourcentage affiché sans source aurait été une information inventée.

Réponse technique. Je conserve un état unavailable/null et une personnalisation facultative de la fiche, sans protocole constructeur non documenté.

Vérification. Tests sur récepteur sans batterie et validations de plages.

Ce que je retiens. La lecture nécessite une source publiée ou accessible par Windows.

La fréquence d'écran n'était pas un entier attendu

Contexte et problème. Le matériel affichait une fréquence fractionnaire différente d'une valeur marketing arrondie.

Analyse. Une conversion forcée en 240 effacerait la précision de Windows.

Réponse technique. Lecture de la fréquence rationnelle réellement active via les APIs d'affichage.

Vérification. Test documenté à 239,765 Hz puis 241 Hz et fixtures de fréquences fractionnaires.

Ce que je retiens. Une fréquence active ne décrit pas toutes les limites de la dalle.

WMI ne renvoyait pas toujours les disques

Contexte et problème. L'inventaire matériel devait résister à un refus WMI.

Analyse. Une unique source peut être partielle selon les permissions et pilotes.

Réponse technique. Fallbacks de stockage en lecture seule et rapprochement volumes/disques.

Vérification. Essai natif documenté sur deux volumes et deux disques.

Ce que je retiens. La santé SMART détaillée n'est pas promise.

Le CPU montrait une valeur incohérente après la veille

Contexte et problème. Un calcul de différence peut mélanger deux périodes discontinues.

Analyse. Les compteurs cumulés nécessitent un intervalle de mesure valide.

Réponse technique. Réinitialisation des références CPU et réseau au réveil, bornage des valeurs.

Vérification. Tests de delta, de réinitialisation et de suspension.

Ce que je retiens. Une métrique indisponible vaut mieux qu'une pointe de graphe artificielle.

La lecture de paramètres était risquée

Contexte et problème. Un objet de préférences non validé pouvait contenir des champs inconnus ou trop volumineux.

Analyse. Les données locales sont une entrée non fiable au même titre qu'un message IPC.

Réponse technique. Liste de clés autorisées, validations de formes et écriture atomique sérialisée.

Vérification. Tests de préférences, audio et persistance.

Ce que je retiens. Les modifications futures des schémas nécessitent une stratégie de migration.

Le build Windows échouait malgré la compilation

Contexte et problème. La chaîne electron-builder fabriquait l'installateur, puis essayait de publier automatiquement sur GitHub sans GH_TOKEN.

Analyse. Une compilation d'artefact et une publication de release sont deux opérations distinctes.

Réponse technique. Le workflow a été corrigé avec npm run dist -- --publish never ; les exécutables sont ensuite uploadés comme artefacts Actions.

Vérification. Le workflow Windows a réussi le 30 octobre 2026 et un artefact Orbit-Windows a bien été trouvé.

Ce que je retiens. Les exécutables ne sont pas signés ; succès du build ne signifie pas validation pour tous les utilisateurs.

12 Tests et résultats sans surinterprétation

La validation documentée

Le rapport du 30 octobre 2026 indique **22 tests unitaires réussis sur 22** lors de la préparation de la source, un typecheck, un lint et des tests natifs du bridge. Les GitHub Actions récents ont également réussi la compilation Windows.

Je distingue trois niveaux de preuve : un test unitaire sur de fausses données, un test natif sur un PC précis et une compilation qui produit des exécutables. Aucun de ces résultats ne garantit que le capteur d'un autre constructeur sera accessible.

Vérification	Périmètre démontré
Tests TypeScript/Node	Calcul CPU, réseau, normalisation, état des métriques, IPC, timer, préférences.
Tests natifs	Dialogue avec OrbitBridge.exe et informations lues sur un environnement Windows.
Windows build CI	Construction du code, packaging NSIS/portable et conservation d'artefacts.
Playwright préparé	Scénarios desktop à exécuter sur une machine réellement configurée.
Futurs essais matériels	Multi-GPU, plusieurs écrans, batteries constructeur et périphériques divers.

Exemples de tests que je sais expliquer

- CPU : un compteur sans delta valide doit produire une mesure indisponible plutôt qu'un pourcentage négatif.
- Réseau : si un compteur baisse après reconnexion, aucun débit négatif ne doit être calculé.
- Périphériques : deux appareils portant le même nom ne doivent pas être confondus par défaut.
- Audio : un volume à 101 % ou une commande contenant une clé inconnue doit être refusé.
- Sécurité : une URL IPC d'une autre origine ou une traversée de chemin doit être rejetée.
- Persistance : un fichier écrit puis relu conserve des préférences valides.
- Focus : une échéance expirée doit terminer le minuteur une seule fois.
- GPU : une association ambiguë ne doit pas réattribuer arbitrairement un compteur à une carte.

Mesurer sans confondre benchmark et marketing

Un benchmark court du collecteur est documenté après échauffement, avec un intervalle rapide de mesure. J'en retiens les enjeux de cadence et de consommation, mais je ne présenterais pas ce résultat comme une preuve universelle d'impact « nul » sur la batterie ou les performances de tous les PC.

La validation de distribution reste distincte : installation et désinstallation sur compte non administrateur, antivirus, vérification du binaire, signature éventuelle et comportement en sommeil réel.

13 Construire et distribuer une application Windows

Chaîne de compilation

Les scripts **build-native.ps1** et **build.mjs** compilent respectivement les parties natives et l'interface. **electron-builder.yml** organise les ressources et les cibles Windows. Le produit assemble le code JavaScript dans app.asar, tandis que le collecteur C# et les ressources nécessaires sont copiés dans les dossiers prévus de l'application.

Cette séparation évite de dépendre de chemins absolus vers mon dossier de développement. Un exécutable livré doit pouvoir trouver ses fichiers en dehors du dépôt Git.

```
npm ci
npm run typecheck
npm run lint
npm test
npm run native && npm run test:native
npm run build
npm run dist -- --publish never
-> installateur NSIS et version portable (non signés)
```

Les avantages des artefacts CI

Le workflow GitHub Actions pour Windows lance les tests et le packaging puis conserve les exécutables comme artefacts. Une vérification récente a confirmé l'existence d'un artefact Orbit-Windows correspondant au build réussi.

Je distingue les artefacts de CI, adaptés à une validation interne, des GitHub Releases destinées à la diffusion. Le dépôt ne doit pas stocker systématiquement les exécutables compilés dans son historique.

Risques avant mise à disposition publique

- Tester sur Windows 11 standard, sans privilèges élevés inutiles.
- Vérifier l'origine du build et envisager une signature Authenticode pour une diffusion large.
- Contrôler les licences Electron, React, bibliothèques et ressources visuelles.
- Confirmer les performances lorsque le collecteur est inaccessible ou partiellement disponible.

- Vérifier la compatibilité multi-écrans, réseau déconnecté, session de veille et mises à jour de Windows.

14 Bilan personnel et décisions d'ingénierie

Les décisions que j'assume

Je retiens surtout la séparation entre l'interface agréable à utiliser et les mécanismes système que je veux garder contrôlables. Electron permet d'unifier l'expérience desktop et les composants React ; C# me donne accès à des APIs Windows sans exposer ces privilèges directement à la page.

Je n'ai pas cherché à contourner les limites de Windows pour afficher toutes les statistiques imaginables. Le produit doit admettre les valeurs indisponibles, limiter les requêtes natives et préserver des données locales potentiellement sensibles.

Ce que je ferais ensuite

Priorité	Étape défendable et mesurable
1	Faire une matrice de tests sur plusieurs PC Windows avec versions, pilotes et périphériques différents.
2	Exécuter et documenter la suite E2E Electron/Playwright sur une machine équipée.
3	Mesurer la consommation CPU/RAM en arrière-plan sur plusieurs durées.
4	Consolider la distribution : signature, Release, install/désinstall, provenance des artefacts.
5	Améliorer la présentation des sources et états des capteurs sans promettre une donnée non accessible.

Ce que le projet m'enseigne

Un logiciel de diagnostic est un excellent exercice de programmation système : la difficulté ne tient pas à afficher une valeur, mais à **savoir d'où elle vient, quand elle est pertinente, ce qu'elle représente et quels droits sont nécessaires pour l'obtenir**.

Orbit m'oblige à raisonner sur le temps, la concurrence, les interfaces de processus, l'hétérogénéité matérielle, la sécurité du navigateur embarqué et les conditions de distribution. Ce sont des compétences plus profondes que le rendu esthétique de la fenêtre.

15 Transformer le besoin en exigences vérifiables

Une application de supervision n'est utile que si l'utilisateur comprend d'où vient chaque mesure, à quel moment elle a été prise et ce qui demeure hors de portée des interfaces Windows.

Des exigences à la fois fonctionnelles et négatives

Le besoin initial associait deux attentes : disposer d'un logiciel Windows réellement installé, au design soigné, et comprendre son matériel sans multiplier les utilitaires. À cette promesse positive, j'ajoute une exigence tout aussi importante : ne jamais fabriquer une température, une capacité de batterie ou un modèle d'écran lorsqu'aucune source fiable ne les expose.

La collecte ne doit pas monopoliser les ressources de l'ordinateur qu'elle observe. Les fréquences de sondage, la visibilité de la fenêtre et le mode économie doivent donc agir sur le rythme du moniteur. Enfin, le navigateur de rendu ne doit pas pouvoir lancer librement des commandes Windows : les interfaces React restent distinctes des API natives.

Question utilisateur	Source / traitement	État de repli
Quel processeur est installé ?	Inventaire natif Windows et normalisation	Inconnu si source absente
Quelle est l'activité CPU ?	Différence de compteurs sur un intervalle	Aucune mesure si delta invalide
Quels écrans sont raccordés ?	Inventaire système et propriétés d'affichage	Détails partiels
Ma souris a-t-elle une batterie ?	Données HID ou interfaces fournisseurs	Non exposé
Quel débit réseau est observé ?	Différences de compteurs et temps écoulé	Non disponible si compteur réinitialisé

Ne pas confondre observation et interprétation

Une quantité d'octets reçus depuis le démarrage n'est pas un débit. Une fréquence nominale n'est pas une fréquence instantanée. Une valeur d'utilisation CPU ne décrit pas nécessairement la charge d'un cœur en l'absence d'un intervalle temporel. Je définis donc pour chaque métrique son unité, sa source, son sens et son état de validité. Cette définition constitue le contrat entre le moteur de collecte et l'affichage.

16 Contrats TypeScript : rendre l’incertitude explicite

Une valeur doit porter son contexte

shared/contracts.ts formalise les objets transmis entre les couches et shared/logic.ts propose une fonction metric qui enveloppe la valeur avec une source, un état et éventuellement une unité. L’absence de donnée est représentée par null et par un état intelligible. Cette convention évite qu’un graphique interprète une valeur manquante comme un zéro mesuré, ou qu’une température non fournie devienne arbitrairement normale.

TypeScript n’empêche pas un processus natif de fournir une chaîne ou un nombre hors bornes. Le code de normalisation reste donc nécessaire : shared/logic.ts valide les pourcentages, les préférences, les opérations audio et les intervalles. system/collectors/normalize.ts accepte des données brutes parfois incomplètes et reformule une représentation cohérente pour les pages React.

```
Exemple conceptuel d’une mesure :
valeur = null
source = "WMI"
etat = "unsupported"
unite = "°C"

Règle d’affichage :
ne pas transformer null en 0 ; afficher la limite de la source
```

Pourquoi faire entrer les erreurs dans le modèle

J’ai intérêt à distinguer un périphérique non supporté, un appel expiré, un défaut de permission et une absence momentanée de mesure. Ces cas entraînent des décisions UX différentes : proposer une action, réessayer, laisser une mesure en attente ou afficher une impossibilité durable. Les transmettre depuis la couche de collecte limite les interprétations fragiles dans le composant graphique.

La précision du contrat améliore également les tests. Un test peut vérifier qu’un capteur absent fournit un état unsupported plutôt qu’une température par défaut. On obtient ainsi une propriété vérifiable et non une simple préférence de présentation.

17 Collecte native C# et supervision du processus

Pourquoi une couche native plutôt que des commandes arbitraires

Le dossier system/native rassemble des collecteurs C# organisés par domaine : inventaire, CPU, GPU, écrans, stockage, réseaux Wi-Fi, périphériques, audio et informations SMBIOS. Ce choix offre un point d'accès explicite aux API Windows et évite de disperser des commandes shell dans chaque page de l'application.

Le bridge TypeScript system/providers/bridge.ts contrôle le lancement du programme natif et son temps d'exécution. Il doit pouvoir gérer l'absence de l'exécutable, un délai dépassé, un code de retour invalide ou une sortie inexploitable. L'échec d'un collecteur ne doit pas interrompre la lecture des autres familles de mesures : c'est un principe de dégradation partielle plutôt que de panne globale.

Une panne de capteur n'est pas une panne d'application

tests/bridge.test.ts vérifie notamment qu'un collecteur manquant provoque un rejet borné sans processus orphelin et qu'un échec natif isolé laisse disponibles les informations CPU et RAM de repli. Il est important d'expliquer ce résultat avec précision : il démontre le comportement des doubles de test, pas la compatibilité exhaustive sur toutes les machines Windows.

Le processus natif doit être considéré comme une dépendance susceptible d'échouer, au même titre qu'une API distante. La différence est qu'il se trouve sur le poste utilisateur : on peut en maîtriser davantage le protocole, mais on doit encore protéger les chemins, les paramètres, les permissions et les sorties.

Frontière	Entrée contrôlée	Erreur à prévoir
React vers preload	Canal d'application autorisé	Paramètre mal formé
Preload vers Electron main	Origine et fenêtre reconnues	Tentative IPC étrangère
Electron vers bridge	Commande native prédéfinie	Timeout ou exécutable absent
Bridge vers collecteur C#	Arguments fermés et format attendu	Sortie non valide
Normalisation vers interface	Données typées avec états	Valeurs partielles ou inconnues

18 Séries temporelles et performance du moniteur

Échantillonnage et notion de delta

Mesurer l'utilisation CPU consiste à comparer deux observations de compteurs plutôt qu'à afficher une valeur isolée. La fonction cpuDelta de shared/logic.ts calcule les différences de temps d'activité et d'inactivité. Un intervalle non positif ou des compteurs incohérents doivent être écartés, faute de quoi le résultat pourrait être supérieur à 100 pour cent ou décrire une situation qui n'a jamais existé.

Le réseau repose sur une logique analogue : à partir de deux valeurs cumulées de réception et d'envoi, la différence divisée par le temps donne un débit. Une réinitialisation de compteur, une interface désactivée ou un changement de réseau impose une prudence particulière. Les trous de données sont plus honnêtes qu'une courbe artificiellement continue.

Un mode économie fonctionnel

La fonction effectiveInterval tient compte de la visibilité, des préférences d'économie et d'une suspension éventuelle. Cette approche évite de laisser tourner au même rythme les collecteurs et les graphiques lorsqu'aucun utilisateur ne regarde l'application. C'est une décision à double effet : elle diminue la charge et évite d'écrire inutilement des historiques sur disque.

Le code Electron conserve des historiques conditionnels et contrôle l'intervalle d'écriture. Je dois distinguer la fréquence de collecte, la fréquence de rendu et la fréquence de persistance : elles ne devraient pas toujours être identiques. Sinon, une interface à 60 images par seconde pourrait multiplier des accès coûteux au matériel sans produire une information plus récente.

Élément	Variable importante	Erreur évitée
CPU	Intervalle entre deux lectures	Pourcentage faux
Réseau	Octets delta / secondes	Confusion volume et débit
Historique	Rétention et taille maximale	Fichiers croissants
UI	Nombre de re-rendus	Charge graphique inutile
Fenêtre masquée	Mode de sondage	Collecte inutile
Mesure absente	État null / indisponible	Zéro fictif

19 Périphériques réels : identification imparfaite et regroupement

Pourquoi un dongle n’est pas forcément le périphérique

Un clavier ou une souris sans fil relié par dongle 2,4 GHz peut apparaître comme plusieurs interfaces HID génériques. Windows expose alors le récepteur, des composants clavier et souris ou des services annexes, sans fournir nécessairement le modèle commercial exact ni une information de batterie exploitable. La présence d’un identifiant n’implique donc pas une identification fiable.

shared/logic.ts regroupe les périphériques à partir de leurs propriétés et choisit des libellés significatifs plutôt que de présenter plusieurs lignes abstraites. La logique favorise les noms moins génériques et rassemble les attributs de plusieurs interfaces. Si une mesure de batterie n’est pas publiée, elle reste explicitement non disponible. Le fait que l’utilisateur voie un périphérique connecté n’autorise pas l’interface à deviner sa charge restante.

Les compromis du regroupement heuristique

Un regroupement incorrect peut fusionner deux appareils distincts ou laisser plusieurs lignes représentant un seul objet. C’est pourquoi je documente le caractère heuristique de cette étape et prévois de confronter les résultats à des configurations matérielles variées. L’algorithme permet d’améliorer l’affichage, pas de promettre une précision universelle.

Le même raisonnement s’applique aux écrans, volumes et disques : il faut rapprocher des inventaires de sources parfois différentes, en évitant de déduire une relation lorsqu’il existe plusieurs candidats indistinguables. Le code de normalisation utilise des correspondances et ne complète certaines informations qu’en présence de liens suffisamment nets.

> **Critère d’honnêteté technique**
>
> Je préfère présenter « non exposé par Windows ou par ce périphérique » plutôt qu’une valeur estimée sans fondement. Dans un outil de diagnostic, l’incertitude fait partie du résultat.

20 Menaces propres aux applications Electron

Le renderer comme zone à privilèges réduits

Electron assemble une technologie web et un environnement desktop. Le risque principal apparaît si une page ou un composant de rendu peut accéder directement au système de fichiers, aux processus ou aux API natives. Le dépôt utilise une séparation entre le renderer, le script preload et le processus principal, avec des canaux IPC explicitement autorisés.

shared/security.ts contient assertAppFrame et rendererFile. Ces fonctions vérifient notamment l'origine, le fait qu'un appel provienne de la frame principale et la possession de la fenêtre, ainsi que les chemins autorisés. tests/security.test.ts couvre des appels malveillants ou des chemins de ressources non reconnus. Ces tests n'éliminent pas toutes les attaques, mais donnent un périmètre d'autorité explicite.

Défense en profondeur et risques résiduels

J'ai intérêt à combiner isolation de contexte, sandbox, désactivation de l'intégration Node pour le renderer, CSP et validation des paramètres. Une seule barrière ne remplace pas les autres : une donnée JSON malformée doit être rejetée par la couche qui l'utilise, même si l'interface a déjà validé un champ. Les paramètres de préférences, de sortie audio ou d'ouverture d'une vue ne doivent pas devenir des commandes système implicites.

Les scripts natifs et les binaires distribués forment également une chaîne de confiance. L'application doit éviter les chemins arbitraires, les exécutables téléchargés silencieusement et les dépendances non vérifiées. La signature de code reste un objectif distinct de la simple production d'un fichier exécutable.

Risque	Frontière critique	Protection présente ou attendue
Renderer non fiable	UI vers système	Preload à API limitée et validation IPC
Navigation vers origine distante	Fenêtre Electron	Restriction d'origine et de contenu
Paramètre injecté	IPC vers natif	Validation de formes et valeurs
Ressource locale arbitraire	Protocole de fichiers	Chemins autorisés
Dépendance compromise	Construction et livraison	Lockfile, tests, revue, signature future

21 Construction Windows : de la source à un artefact utilisable

Les différentes étapes d'une livraison

Le projet contient des scripts dédiés au développement, à la compilation du collecteur C#, à la vérification native et au packaging Electron. Dans le workflow windows.yml, GitHub Actions réalise la validation TypeScript, le lint, les tests, la compilation native et la génération des exécutables. La création d'un artefact permet ensuite de récupérer la sortie de compilation sans avoir à installer immédiatement un système de publication automatique.

Un incident documenté illustre la nécessité de distinguer compilation et publication. Une première exécution avait produit des fichiers Windows mais échouait ensuite parce que electron-builder tentait de les publier via GitHub sans jeton dédié. Le correctif a consisté à désactiver la publication implicite avec --publish never. Les exécutions suivantes ont réussi ; ce changement ne modifie pas l'application exécutée, mais rend le pipeline de livraison conforme à son objectif.

Ce qu'un résultat CI positif ne prouve pas

Un exécutable compilé sur le runner Windows prouve que la chaîne de construction a fonctionné avec ses dépendances. Cela ne démontre ni l'absence de faux positifs antivirus, ni la stabilité d'un capteur particulier, ni la compatibilité avec des périphériques 2,4 GHz spécifiques. Le workflow doit être présenté comme une étape de validation, complétée ensuite par des tests d'installation, de désinstallation, de démarrage et de fonctionnement sur postes réels.

Dans une logique de distribution, je voudrais aussi définir le versionnage, les notes de changement, les signatures de binaires, les sommes de contrôle et un mécanisme de retour à une version précédente. Ces éléments ne sont pas tous présents aujourd'hui : ils constituent des suites mesurables, sans confusion avec le travail déjà effectué.

Étape	Preuve attendue	Limite
npm ci ou installation	Dépendances disponibles	N'assure pas la sécurité de tous les paquets
Typecheck / lint	Contrats et règles statiques	Ne prouve pas le comportement
Tests Node / C#	Logique locale vérifiée	Matériel physique non exhaustif
Packaging Electron	Exécutable produit	Pas une release certifiée
Artefact GitHub	Fichier récupérable	Durée de conservation limitée

22 Programme d’essais et journal de décisions

Valider des configurations réelles, pas seulement une machine

Une grille d’essais utile couvre des machines avec GPU intégré seul, GPU dédié, écran unique, double écran, différents pilotes audio, Wi-Fi, réseau câblé, souris USB, périphérique Bluetooth et dongle propriétaire. Pour chaque configuration, je consignerais les mesures attendues, les valeurs réellement disponibles, les écarts, le coût CPU du moniteur et les informations identifiantes à ne pas exporter. Ce protocole peut conduire à améliorer la normalisation plutôt qu’à ajouter un correctif particulier à un seul matériel.

Je séparerais les anomalies du collecteur et celles de l’interface. Une donnée correcte mais mal arrondie relève du rendu. Une donnée absente dans la réponse native appelle une investigation sur la source Windows. Une donnée jamais exposée par le système n’est pas nécessairement un bug d’Orbit : le bon résultat peut être un état d’indisponibilité.

Récapitulatif des choix techniques

Arbitrage	Avantage recherché	Coût / point de vigilance
Electron et React	UI desktop évolutive	Surface de sécurité à réduire
Collecteurs C# séparés	Accès précis aux API Windows	Packaging natif à maîtriser
Contrats typés	Sources et états explicites	Normalisation indispensable
Mesures nullables	Pas de fausse précision	Interfaces adaptées aux états absents
Sondage adaptatif	Charge réduite	Latence variable selon visibilité
Artifacts sans publication	CI autonome	Distribution officielle à organiser

Épreuve orale

Je peux montrer un même problème sous trois angles : sa cause observée dans le collecteur, sa représentation normalisée en TypeScript et son effet visuel. Cette chaîne prouve mieux une compréhension technique qu’une simple démonstration de captures.

23 Questions d'oral et réponses argumentées

Question possible : Pourquoi Electron plutôt qu'un navigateur ouvert sur localhost ?

Réponse que je peux défendre : Je voulais une application de bureau avec une fenêtre indépendante, une distribution Windows et un contrôle explicite des ressources natives. J'ai donc remplacé le serveur HTTP par Electron et une frontière IPC.

Question possible : Pourquoi une application C# à côté de TypeScript ?

Réponse que je peux défendre : C# permet d'appeler des APIs Windows et des interfaces natives dans un composant dédié. Le frontend n'a ainsi pas besoin de privilèges généraux pour ses composants UI.

Question possible : Comment obtenez-vous une mesure CPU ?

Réponse que je peux défendre : Par différence entre deux échantillons cumulés : activité approximativement égale à 1 moins la fraction de temps idle, sous réserve d'un intervalle valide.

Question possible : Pourquoi une souris 2,4 GHz n'affiche-t-elle pas forcément sa batterie ?

Réponse que je peux défendre : Le dongle ou l'interface HID peut ne pas fournir de champ Battery Strength public. Je refuse d'inventer une valeur et affiche la limite du fournisseur.

Question possible : Quelles protections Electron ont été retenues ?

Réponse que je peux défendre : Isolation de contexte, sandbox, intégration Node désactivée, origine locale, preload à API limitée et vérification de l'origine et de la frame à chaque IPC.

Question possible : Comment gérez-vous une source WMI indisponible ?

Réponse que je peux défendre : En utilisant d'autres APIs publiques quand cela est prévu et en propageant un état d'indisponibilité lorsqu'aucune mesure valide n'existe.

Question possible : Comment les graphiques évitent-ils de mentir ?

Réponse que je peux défendre : Les valeurs nulles ne sont pas remplacées par zéro et les trous ne sont pas interpolés comme des observations réelles.

Question possible : Pourquoi le build GitHub échouait-il ?

Réponse que je peux défendre : La compilation des exécutables réussissait, mais electron-builder essayait d'enchaîner une publication nécessitant un jeton. La CI désactive maintenant la publication implicite et stocke les exécutables en artefacts.

Question possible : Quel est le principal travail restant ?

Réponse que je peux défendre : La validation E2E et matérielle sur un parc plus diversifié, puis une distribution signée et contrôlée.

24 Annexes techniques et documentation

Carte de navigation des sources

Source	Ce que j'y vérifie
docs/AUDIT.md	Migration documentée depuis le prototype web initial.
docs/ARCHITECTURE.md	Séparation Electron, preload, collecte et cadence.
shared/logic.ts	cpuDelta, networkRates, groupDevices, préférences et états de métriques.
shared/security.ts	Contrôles de provenance IPC et limitation des chemins de rendu.
system/monitor.ts	Collecte rapide et inventaire, verrous et économie en arrière-plan.
system/native/*.cs	APIs audio, affichage, stockage, GPU, capteurs, SMBIOS, Wi-Fi et périphériques.
electron/main/*.ts	Gestion des fenêtres, diagnostics et préférences atomiques.
src/pages/*.tsx	Navigation et présentations spécialisées par catégorie.
tests/*.test.ts	Tests de logique, sécurité, bridge, collecteurs et persistance.
.github/workflows/windows.yml	Compilation CI Windows et artefacts installables et portables.

Glossaire personnel

Terme	Définition utile
IPC	Communication entre processus, avec canaux et paramètres contrôlés.
Preload	Script Electron exposant au renderer une API autorisée et restreinte.
SMBIOS	Informations matérielles structurées fournies par le firmware.
WMI	Infrastructure Windows d'inventaire et d'administration.
DXGI / NVML	Interfaces pour certaines informations GPU et relevés NVIDIA.
PDH	Compteurs de performance Windows.
HID / ContainerId	Descriptions d'interfaces de périphériques et regroupement logique.
Null / indisponible	Absence de mesure démontrable ; différent d'une valeur égale à zéro.
Idempotence	Une action rejouée sans effet supplémentaire non désiré.
Packaging	Assemblage du produit en installateur ou application portable.

Sources et portée des déclarations

Dépôt public : https://github.com/carlxwj/orbit-windows ; documents vérifiés : docs/AUDIT.md, docs/ARCHITECTURE.md, docs/VALIDATION.md, docs/HARDWARE.md, docs/SECURITE_DONNEES.md et docs/DEPLOIEMENT.md. Les observations matérielles chiffrées correspondent au rapport daté du 30 octobre 2026, non à une mesure nouvellement effectuée pour ce PDF.

Ce dossier explique l'architecture effectivement lisible dans le code et les problèmes explicitement documentés. Il ne constitue pas une attestation de développement exclusif, un audit de sécurité indépendant ou une certification de compatibilité matérielle. La provenance de la donnée et les limites de validation font partie de la qualité technique du projet.